Live Coding with AI

Building Real Solutions with Terminal Agentic Tools

Claude Code · Codex · OpenCode · and more →

Table of Contents

1. Live Coding with AI
2. What are Terminal Agentic Tools?
3. The Agent Loop
4. Install Claude Code
5. Prompted to log in on first use
6. Claude Code – Key Commands
7. Claude Code – Permission Model
8. OpenAI Codex CLI
9. OpenCode
10. Other Tools Worth Knowing
11. Tool Comparison
12. When to Use Which
13. Prompting Patterns That Work
14. Real-World Workflows
15. Key Takeaways

What are Agentic

Coding Tools?

What are Terminal Agentic Tools?

AI assistants that **live in your terminal** – they don't just suggest code, they *act*.

🤔

Traditional AI

You copy-paste suggestions from a chat UI into your editor

⚡

Agentic Tools

The AI reads your files, writes code, runs tests – autonomously

🔁

The Loop

Plan → Act → Observe → Repeat until the task is done

You describe the goal. The agent figures out the steps.

Meet

Claude Code

Anthropic's official terminal coding agent

Install Claude Code

Pick your platform — one command and you're running.

```
# ── WinGet (Windows) ──────────────────────────────
# Does NOT auto-update — run upgrade periodically
winget install Anthropic.ClaudeCode

# Upgrade when needed
winget upgrade Anthropic.ClaudeCode

# Then start Claude Code
claude
```

- macOS / Linux / WSL
- Windows PowerShell
- Windows CMD
- Homebrew
- WinGet

💡 Native installs (curl/irm) auto-update silently

🍺 Homebrew / WinGet require manual `upgrade`

🌐 Also: Web · Desktop App · VS Code · JetBrains · Slack · GitHub Actions

Getting Started — First Session

Log in once, then describe what you want in plain English.

Step 1 — Log in

```
claude
# Prompted to log in on first use

/login
# Switch accounts at any time
```

Supports: Claude Pro/Max/Teams/Enterprise · Console API · Bedrock · Vertex AI · Foundry

Step 2 — Start in your project

```
cd /path/to/your/project
claude
```

Step 3 — Ask anything

```
what does this project do?
what technologies does this project use?
```

What it can do

- ✓ Read, write, and edit any file in your project
- ✓ Run shell commands, tests, and build scripts
- ✓ Search your codebase with Glob and Grep tools
- ✓ Create git commits, branches, and PRs via `gh`
- ✓ Spawn parallel sub-agents for independent tasks
- ✓ Connect to MCP servers for external integrations

CLAUDE.md — Project Memory

```
# My Project

## Stack
- Next.js 15, TypeScript, Tailwind
- Supabase for auth + database

## Conventions
```

Claude Code – Key Commands

```
# ── Interactive sessions ──────────────────────────
claude                          # start session in cwd
claude "fix the failing tests"  # session with initial prompt
claude -c                       # continue most recent conversation

# ── One-shot / scripted ───────────────────────────
claude -p "explain this function"
claude -p "list all TODO comments" --output-format json
echo "refactor auth.ts" | claude --print
cat error.log | claude -p "diagnose and fix"

# ── Resume conversations ──────────────────────────
claude -r                       # pick from recent sessions
claude commit                   # create a git commit

# ── Inside a session ──────────────────────────────
/help      /clear      /login
/compact   /memory     /doctor
/review    /bug        /pr_comments
```

⌨️ Press `?` for keyboard shortcuts · Tab for completion · ↑ for history

🔁 Type `/` to browse all commands and installed skills

Claude Code – Permission Model

Claude asks before taking risky actions. You control the blast radius.

Auto-allowed

- Read files
- Search (Glob/Grep)
- Run tests
- Git status/diff/log

Asks first

- Write / edit files
- Install packages
- Run shell commands
- Git commit

Always confirms

- Git push / force push
- Delete files/branches
- Destructive operations
- External API calls

```
# Allow specific tools without prompting (session flag)
claude --allowedTools "Bash(npm run test),Edit,Read"

# Or lock it down in ~/.claude/settings.json
# "permissions": { "allow": [ ... ], "deny": [ ... ] }
```

The Ecosystem

Other Terminal Agents

OpenAI Codex CLI

OpenAI's terminal agent — multimodal and sandboxed.

```
npm install -g @openai/codex
codex "migrate this Express app to Fastify"
```

Key features

- Multimodal: pass screenshots or diagrams
- Sandboxed execution (macOS Seatbelt / Docker)
- Supports `o4-mini`, `o3` models
- Full-auto mode with network-disabled sandbox

Approval modes

```
# Suggest only — you review every change
codex --approval-mode suggest "..."

# Auto-edit — edits files, asks before running
codex --approval-mode auto-edit "..."

# Full-auto — completely autonomous
codex --approval-mode full-auto "..."
```

Config: `~/.codex/config.yaml` or `AGENTS.md` in project root

OpenCode

An open-source TUI (terminal UI) agentic coding tool.

```
# Install via Go or Homebrew
brew install opencode-ai/tap/opencode
opencode
```

What makes it different

- Rich TUI with split panes (chat + file diff)
- Provider-agnostic: Claude, GPT-4, Gemini, local LLMs
- LSP integration for context-aware edits
- 100% open source (MIT) – run fully offline

opencode.json config

```
{
  "model": "anthropic/claude-sonnet-4-6",
  "autoapprove": false,
  "keybindings": {
    "submit": "enter",
    "new_session": "ctrl+n"
  },
  "mcp": {
    "servers": {}
  }
}
```

Great for: teams who want model flexibility without vendor lock-in

Other Tools Worth Knowing

🤖 Aider Open Source

Python-based, git-native agent. Tracks changes with git automatically.

```
pip install aider-chat && aider
```

🌊 Amp (Sourcegraph) Terminal

Deep codebase search + agent from Sourcegraph team. Strong at large repos.

```
npm install -g @sourcegraph/amp
```

🦀 Gemini CLI Google

Google's terminal agent. 1M token context window. Free tier available.

```
npm install -g @google/gemini-cli &&
gemini
```

🧠 Goose (Block) Open Source

Extensible agent from Block (Square). Plugin system for custom tools.

```
brew install block/tap/goose
```

How do they

Stack Up?

Tool Comparison

	Claude Code	Codex CLI	OpenCode	Aider	Gemini CLI
Model	Claude (Anthropic)	GPT / o-series	Any provider	Any provider	Gemini
UI	Terminal REPL	Terminal REPL	TUI (split panes)	Terminal REPL	Terminal REPL
Open Source	❌	❌	✅	✅	❌
MCP Support	✅	❌	✅	❌	✅
Sandbox	Permissions model	macOS / Docker	❌	❌	❌
Project config	`CLAUDE.md`	`AGENTS.md`	`opencode.json`	`.aider.conf.yml`	`GEMINI.md`

When to Use Which

Claude Code

Best all-rounder. Strongest at complex multi-file refactors, PR workflows, and MCP integrations. Best-in-class reasoning.

Codex CLI

Reach for it when you need sandboxed execution or multimodal input (paste a screenshot of a UI to build).

OpenCode

When your team needs model flexibility or you want 100% open-source, self-hostable tooling.

Gemini CLI

When context size matters — ingest an entire large codebase in one shot with 1M token window.

The honest take

- They are converging rapidly — features copy across tools weekly
- The model matters more than the shell around it
- Your **project config file** is the biggest productivity multiplier

Pick by team context

🏢 Enterprise with Anthropic contract → Claude Code
🔒 OSS / self-host required → OpenCode + Aider
📐 Need multimodal → Codex CLI
📚 Huge legacy codebase → Gemini CLI

Patterns for

Live Coding

Prompting Patterns That Work

Writing good prompts is the highest-leverage skill.

❌ Vague

```
make this code better
```

✅ Specific + Constrained

```
refactor auth.ts to use async/await.
Keep the existing function signatures.
Run npm run typecheck when done.
```

❌ Too broad

```
build me an e-commerce site
```

✅ Scoped + Verifiable

```
add a /cart route to the Express API
that accepts { productId, qty } and
```

Pattern: Anchor to files

Reference specific files and line numbers – agents explore better when they have a starting point rather than scanning everything.

Pattern: Specify the verification step

Tell the agent how to confirm success: *"run npm test and make sure all tests pass"*. This closes the feedback loop.

Pattern: Use CLAUDE.md as a multiplier

Every convention you document there is a prompt you never have to write again. Think of it as onboarding docs for the AI.

Real-World Workflows

🔧

Bug Fix Loop

1. Paste error log / failing test
2. Agent traces root cause
3. Proposes + applies fix
4. Re-runs test to verify
5. Creates commit message

🏗️

Feature Scaffold

1. Describe the feature + constraints
2. Agent proposes a plan first
3. You approve / adjust the plan
4. Agent implements incrementally
5. Review diffs in your editor

🔄

Codebase Migration

1. Define the target state in CLAUDE.md
2. Agent inventories affected files
3. Migrates in batches
4. Runs typecheck after each batch
5. Creates a PR with a full summary

Pro tip — keep humans in the loop strategically: Let the agent do the boring parts (boilerplate, migrations, test setup). Review architecture and business logic decisions yourself. The agent is fast; you provide judgment.

Key Takeaways

🧠 **Agentic ≠ Autocomplete**
These tools act in a loop – they read, edit, run, and verify until the task is done.

📄 **Config files are leverage**
`CLAUDE.md` / `AGENTS.md` are the highest-ROI thing you can write. Document your conventions once.

🎯 **Specificity beats vagueness**
Anchor prompts to files, define success criteria, and tell the agent how to verify its work.

🔀 **The right tool shifts with context**
No single winner. Claude Code for complex reasoning, Gemini for big context, OpenCode for model freedom.

Start here this week

```
# 1. Install Claude Code
npm install -g @anthropic-ai/claude-code

# 2. Create a CLAUDE.md in your project
echo "# My Project\n## Stack: ..." > CLAUDE.md

# 3. Try a real task
claude "explain what this codebase does
then suggest 3 quick wins for code quality"
```

The best way to learn is to try it on a real problem.

Thank You

Live Coding with AI – Building Real Solutions

Claude Code	Codex CLI	OpenCode	Aider
claude.ai/code	github.com/openai/codex	opencode.ai	aider.chat

Slides built with Slidev